

Diagnóstico Técnico do Projeto

****Projeto:**** goescompracerta

****Data da coleta:**** 2026-09-21 20:35:42 -03:00

****Objetivo:**** fornecer a outro assistente técnico um retrato reproduzível do estado atual do projeto Node.js/React e da integração com Yahoo Finance.

Seção 1: Ambiente

Versões

Comandos executados no terminal do projeto:

```
``text
node -v
v24.19.0
```

```
npm -v
11.17.0
``
```

Pacote Yahoo Finance

Comando executado:

```
``text
npm list yahoo-finance2
``
```

Saída completa:

```
``text
goescompracerta@0.0.0 C:\Users\alexf\OneDrive\Área de Trabalho\goescompracerta
└─ yahoo-finance2@4.0.2
``
```

Dependências de primeiro nível

Comando executado:

```
``text
npm list --depth=0
``
```

Saída completa:

```
```text
```

```
goescompracerta@0.0.0 C:\Users\alex\OneDrive\Área de Trabalho\goescompracerta
```

```
|— @vitejs/plugin-react@4.7.0
|— autoprefixer@10.5.4
|— lucide-react@0.483.0
|— postcss@8.5.26
|— react-dom@18.3.1
|— react@18.3.1
|— recharts@2.15.4
|— tailwindcss@3.4.19
|— vite@5.4.21
|— xlsx@0.18.5
└─ yahoo-finance2@4.0.2
```

```
```
```

Variáveis de ambiente

Foi localizado o arquivo `.env`. Por segurança, o valor não foi incluído:

```
```text
```

```
Arquivo: .env
```

```
Variável identificada: VITE_BRAPI_API_KEY
```

```
Valor: <REDACTED>
```

```
```
```

Observações de instalação

A instalação de `yahoo-finance2` foi concluída. O npm informou 4 vulnerabilidades no conjunto de dependências: 1 moderada e 3 altas. Também informou que há scripts de instalação pendentes de aprovação para `esbuild`.

Seção 2: Estrutura do Projeto

A árvore abaixo foi coletada excluindo `node\_modules`, `dist` e `.git` para evitar conteúdo gerado ou excessivamente grande:

```
```text
```

```
abertura do programa/
```

```
1000131540.mp4
```

```
src/
```

```
components/
```

```
Common/
```

```
KPICards.jsx
```

```
MethodCard.jsx
```

```
Dashboard/
```

```
BenchmarkChart.jsx
```

```
CurrencyWidget.jsx
```

- PortfolioChart.jsx
- PositionSummary.jsx
- Forms/
  - AssetForm.jsx
- Layout/
  - OpeningIntro.jsx
  - Sidebar.jsx
- Portfolio/
  - AssetsTable.jsx
- TresMosqueteirosFII/
- constants/
  - config.js
  - translations.js
- hooks/
  - useAssets.js
  - useProventos.js
  - useTheme.js
  - useUpdateQuotes.js
- pages/
  - TresMosqueteirosFII/
    - TresMosqueteirosFIIPage.jsx
  - CompoundInterestPage.jsx
  - DashboardPage.jsx
  - MasterMethodsPage.jsx
  - PortfolioPage.jsx
  - ProventosPage.jsx
  - RebalancerPage.jsx
  - UserSettingsPage.jsx
- services/
  - assetService.js
  - brapiService.js
  - calculationService.js
  - rebalanceService.js
  - storageService.js
- utils/
  - colors.js
  - cryptoUtils.js
  - formatters.js
  - index.js
- .env
- analise-cotacoes-calculos-eua.html
- ARCHITECTURE.md
- auditoria-benchmark-explicativa.html
- auditoria-benchmark-explicativa.pdf
- CLEAN\_CODE\_ANALYSIS.md
- CLEAN\_CODE\_REPORT.md
- CLEAN\_CODE\_SUMMARY.md

contexto-fiis.md  
DIAGNOSTICO\_BENCHMARK.md  
DOCUMENTATION\_INDEX.md  
explicacao-rebalanceamento-cripto.html  
EXPLICACAO\_BENCHMARK.md  
index.css  
index.html  
main.jsx  
OPTIMIZATION\_GUIDE.md  
package-lock.json  
package.json  
postcss.config.js  
QUICK\_GUIDE.md  
README.md  
REFACTORING\_COMPLETE.md  
REFACTORING\_SUMMARY.md  
regressionTest.mjs  
relatorio-auditoria-calculos.html  
relatorio-auditoria-calculos.pdf  
relatorio-diagnostico-tecnico-matematico-atualizado.html  
relatorio-diagnostico-tecnico-matematico.html  
relatorio-diagnostico-tecnico-matematico.pdf  
relatorio-explicacao-bug-eua.html  
relatorio-explicacao-bug-eua.pdf  
relatorio-integracao-apis.html  
relatorio-integracao-apis.pdf  
relatorio-regressao-final.md  
relatorio-tecnico-preco-medio-custo-total.html  
relatorio-tecnico-preco-medio-custo-total.pdf  
RELATORIO\_TECNICO\_ATUALIZACAO\_COTAS\_BRAPI.md  
server.mjs  
START\_HERE.md  
tailwind.config.js  
VALIDATION\_CHECKLIST.md  
vite.config.js  
yahoo-finance-code.pdf.html  
...

### ## Seção 3: package.json

Conteúdo completo obtido com `Get-Content package.json -Raw`:

```
``json
{
 "name": "goescompracerta",
 "private": true,
 "version": "0.0.0",
```

```

"type": "module",
"scripts": {
 "dev": "node server.mjs",
 "build": "vite build",
 "preview": "vite preview --host 0.0.0.0"
},
"dependencies": {
 "lucide-react": "^0.483.0",
 "react": "^18.3.1",
 "react-dom": "^18.3.1",
 "recharts": "^2.8.0",
 "xlsx": "^0.18.5",
 "yahoo-finance2": "^4.0.2"
},
"devDependencies": {
 "@vitejs/plugin-react": "^4.3.1",
 "autoprefixer": "^10.4.22",
 "postcss": "^10.4.38",
 "tailwindcss": "^3.4.4",
 "vite": "^5.4.1"
}
}
...

```

#### ## Seção 4: Código do arquivo principal

O arquivo principal do backend Node.js é `server.mjs`. O frontend possui `main.jsx` como entrada do Vite, mas o arquivo responsável por iniciar o servidor HTTP e o proxy das APIs é `server.mjs`.

Conteúdo completo de `server.mjs`:

```

``javascript
import { createServer } from 'node:http';
import { spawn } from 'node:child_process';
import { fileURLToPath, URL } from 'node:url';
import YahooFinance from 'yahoo-finance2';

const PORT = Number(process.env.API_PORT || 3001);
const BCB_BASE_URL = 'https://api.bcb.gov.br/dados/serie/bcdata.sgs';
const yahooFinance = new YahooFinance({ suppressNotices: ['yahooSurvey'] });
const ALLOWED_SERIES = new Set(['12', '4390', '4392', '196']);
const MAX_CHUNK_YEARS = 5;

const sendJson = (response, status, payload) => {
 response.writeHead(status, {
 'Content-Type': 'application/json; charset=utf-8',
 'Cache-Control': 'no-store'
 });
 response.write(JSON.stringify(payload));
 response.end();
};

```

```
});
response.end(JSON.stringify(payload));
};
```

```
const parseDate = (value) => {
 if (!/^\\d{2}\\d{2}\\d{4}$/.test(value || '')) return null;
 const [day, month, year] = value.split('/').map(Number);
 const date = new Date(Date.UTC(year, month - 1, day));
 return date.getUTCFullYear() === year
 && date.getUTCMonth() === month - 1
 && date.getUTCDate() === day
 ? date
 : null;
};
```

```
const formatDate = (date) => [
 String(date.getUTCDate()).padStart(2, '0'),
 String(date.getUTCMonth() + 1).padStart(2, '0'),
 date.getUTCFullYear()
].join('/');
```

```
const normalizeYahooSymbol = (value) => String(value || '')
 .trim()
 .toUpperCase()
 .replace(/\\s+/g, '')
 .replace(/\\./g, '-')
 .replace(/-/g, '-');
```

```
const splitDateRange = (startDate, endDate) => {
 const chunks = [];
 let cursor = new Date(startDate);

 while (cursor <= endDate) {
 const chunkEnd = new Date(cursor);
 chunkEnd.setUTCFullYear(chunkEnd.getUTCFullYear() + MAX_CHUNK_YEARS);
 chunkEnd.setUTCDate(chunkEnd.getUTCDate() - 1);
 if (chunkEnd > endDate) chunkEnd.setTime(endDate.getTime());

 chunks.push({
 start: formatDate(cursor),
 end: formatDate(chunkEnd)
 });

 cursor = new Date(chunkEnd);
 cursor.setUTCDate(cursor.getUTCDate() + 1);
 }
}
```

```

 return chunks;
 };

const fetchBcbChunk = async (series, start, end) => {
 const targetUrl = new URL(`${BCB_BASE_URL}.${series}/dados`);
 targetUrl.searchParams.set('formato', 'json');
 targetUrl.searchParams.set('dataInicial', start);
 targetUrl.searchParams.set('dataFinal', end);

 const upstream = await fetch(targetUrl, {
 headers: { Accept: 'application/json' }
 });

 if (!upstream.ok) {
 throw new Error(`Banco Central respondeu HTTP ${upstream.status}`);
 }

 const payload = await upstream.json();
 if (!Array.isArray(payload)) {
 throw new Error('Banco Central retornou JSON inválido');
 }

 return payload;
};

const fetchBcbSeries = async (series, startDate, endDate) => {
 const chunks = splitDateRange(startDate, endDate);
 const responses = await Promise.all(
 chunks.map(({ start, end }) => fetchBcbChunk(series, start, end))
);
 const unique = new Map();

 responses.flat().forEach((item) => {
 if (item?.data && item?.valor !== null) unique.set(item.data, item);
 });

 return [...unique.values()].sort((first, second) => {
 const firstDate = first.data.split('/').reverse().join('-');
 const secondDate = second.data.split('/').reverse().join('-');
 return firstDate.localeCompare(secondDate);
 });
};

const handleYahooRequest = async (request, response) => {
 const requestUrl = new URL(request.url, `http://${request.headers.host}`);
 const symbol = normalizeYahooSymbol(requestUrl.searchParams.get('symbol'));

```

```

if (!symbol || !/^[A-Z0-9.^=-]+$/i.test(symbol)) {
 sendJson(response, 400, { error: 'Ticker Yahoo Finance inválido.' });
 return;
}

try {
 const quote = await yahooFinance.quote(symbol);
 const price = Number(quote?.regularMarketPrice);

 if (!quote || !Number.isFinite(price) || price <= 0) {
 sendJson(response, 404, {
 error: `Ticker ${symbol} não encontrado no Yahoo Finance.`,
 code: 'TICKER_NOT_FOUND'
 });
 return;
 }

 sendJson(response, 200, { symbol, price, currency: quote?.currency || 'USD' });
} catch (error) {
 const message = String(error?.message || 'Falha ao consultar o Yahoo Finance.');
```

```

 const notFound = /not found|invalid symbol|no data found|quote not found/i.test(message);
 sendJson(response, notFound ? 404 : 502, {
 error: notFound ? `Ticker ${symbol} não encontrado no Yahoo Finance.` : message,
 code: notFound ? 'TICKER_NOT_FOUND' : 'UPSTREAM_ERROR'
 });
}
};

const handleBcbRequest = async (request, response) => {
 const requestUrl = new URL(request.url, `http://${request.headers.host}`);
 const series = requestUrl.searchParams.get('series');
 const start = requestUrl.searchParams.get('startDate');
 const end = requestUrl.searchParams.get('endDate');
 const startDate = parseDate(start);
 const endDate = parseDate(end);

 if (!ALLOWED_SERIES.has(series)) {
 sendJson(response, 400, { error: 'Série do Banco Central não permitida.' });
 return;
 }

 if (!startDate || !endDate || startDate > endDate) {
 sendJson(response, 400, { error: 'Datas inválidas. Use o formato dd/MM/yyyy.' });
 return;
 }

 try {

```



```

const data = await fetchBcbSeries(series, startDate, endDate);
if (data.length === 0) {
 sendJson(response, 502, { error: 'Banco Central retornou dados vazios.' });
 return;
}
sendJson(response, 200, { series, startDate: start, endDate: end, data });
} catch (error) {
 sendJson(response, 502, { error: error.message || 'Falha ao consultar o Banco Central.' });
}
};

const server = createServer((request, response) => {
 if (request.method === 'GET' && request.url?.startsWith('/api/yahoo')) {
 handleYahooRequest(request, response);
 return;
 }

 if (request.method === 'GET' && request.url?.startsWith('/api/bcb')) {
 handleBcbRequest(request, response);
 return;
 }

 if (request.method === 'GET' && request.url === '/api/health') {
 sendJson(response, 200, { ok: true });
 return;
 }

 sendJson(response, 404, { error: 'Rota não encontrada.' });
});

server.listen(PORT, '127.0.0.1', () => {
 console.log(`API BCB disponível em http://127.0.0.1:${PORT}`);

 if (process.env.API_ONLY !== '1') {
 const vitePath = fileURLToPath(new URL('./node_modules/vite/bin/vite.js', import.meta.url));
 const vite = spawn(process.execPath, [vitePath, '--host', '0.0.0.0'], {
 stdio: 'inherit',
 env: process.env
 });
 }

 const stop = () => {
 if (!vite.killed) vite.kill();
 server.close() => process.exit();
 };

 process.on('SIGINT', stop);
 process.on('SIGTERM', stop);

```

```
}
});
```
```

Seção 5: Comando de execução

O comando de desenvolvimento definido em `package.json` é:

```
```text  
npm run dev
```
```

Esse script executa:

```
```text  
node server.mjs
```
```

O README também documenta `npm run dev` como o comando de desenvolvimento. O último comando observado no terminal durante esta coleta foi `node regressionTest.mjs`, usado para validação, não para iniciar a aplicação.

Seção 6: Erros observados

Problems do VS Code

A verificação do painel de problemas retornou:

```
```text  
No errors found.
```
```

Terminal

O backend foi iniciado durante a validação e exibiu:

```
```text  
API BCB disponível em http://127.0.0.1:3013
```
```

Não foi observada stack trace ou mensagem de erro no terminal durante a coleta.

Resultado recente da rota Yahoo

A rota `/api/yahoo` foi testada com os ativos principais. O resultado foi:

```
```text
```

```
AAPL: HTTP 200
O: HTTP 200
QQQ: HTTP 200
ROIV: HTTP 200
BRK.B: HTTP 200, normalizado para BRK-B
DOES-NOT-EXIST-XYZ: HTTP 404, código TICKER_NOT_FOUND
...
```

O `404` do ticker inexistente é um erro tratado e esperado, não uma falha não capturada do processo.

#### ### Validações executadas

```
```text
npm run build
vite v5.4.21 building for production...
✓ 2431 modules transformed.
✓ built in 43.97s
...

```

```
```text
node regressionTest.mjs
Cobertura: 29/29 testes passaram.
...

```

#### ### Nota de segurança

Nenhum valor secreto foi incluído neste relatório. A chave do `.env` foi representada apenas pelo nome `VITE\_BRAPI\_API\_KEY`.